Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Cloud Foundations & Virtualization — TP4 — Docker : exécuter, construire, partager et rendre persistants des conteneurs
4e année Cybersécurité — École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
JIDAL Ilyas
KABORE Mohammad Sharif Jonathan

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. KARRAKCHOU & M. SOUIDI

Date : 14 octobre 2026

Table des matières

Cadre du TP **3**

1 Lancer l'image Docker getting-started **4**
1.1 Partie I.1 — Installation de Docker . . . 4
1.2 Partie I.2 — Exécution de l'image getting-started . . . 4
1.2.1 Lancement du conteneur . . . 4
1.2.2 Vérification . . . 5

2 Construire une image Docker **5**
2.1 Partie II.1 — Ouverture du tutoriel . . . 5
2.2 Partie II.2 et II.3 — Lecture de la section Getting Started . . . 6
2.3 Partie II.4 — Section Our Application . . . 6
2.3.1 Récupération du code source . . . 6
2.3.2 Décompression de l'application . . . 6
2.3.3 Création du Dockerfile . . . 6
2.3.4 Construction de l'image . . . 7
2.3.5 Lancement du conteneur de l'application . . . 8
2.3.6 Test de l'application dans le navigateur . . . 9
2.3.7 Contrôle du Dockerfile final . . . 10
2.3.8 Image créée . . . 10
2.4 Partie II.5 — Vérification par l'enseignant . . . 10

3 Mettre à jour une image Docker **10**
3.1 Partie III.1 — Modification du code source . . . 10
3.2 Partie III.2 — Reconstruction de l'image . . . 10
3.3 Partie III.3 — Démarrage d'un nouveau conteneur : l'erreur attendue . . . 11
3.4 Partie III.4 — Remplacement de l'ancien conteneur . . . 12
3.4.1 Arrêt du conteneur . . . 12
3.4.2 Suppression du conteneur . . . 12
3.5 Partie III.5 — Démarrage du conteneur mis à jour . . . 13
3.5.1 Lancement . . . 13
3.5.2 Vérification . . . 13
3.6 Partie III.6 — Contrôle dans le navigateur . . . 13

4 Partager une image Docker **14**
4.1 Partie IV.1 — Dépôt sur Docker Hub . . . 14
4.2 Partie IV.2 — Connexion à Docker Hub . . . 14
4.3 Partie IV.3 — Premier essai de `docker push` : l'échec voulu . . . 15
4.4 Partie IV.4 — Étiquetage de l'image . . . 15
4.5 Partie IV.5 — Envoi de l'image . . . 16
4.6 Partie IV.6 — Exécution sur une nouvelle instance : le playground . . . 16
4.6.1 Démarrage du conteneur dans le playground . . . 16
4.6.2 Exposition du port 3000 . . . 17
4.6.3 Application ouverte depuis le playground . . . 18

5 Rendre persistantes les données créées par un conteneur 18
5.1 Partie V.1 — Le système de fichiers d'un conteneur . . . 18
5.1.1 Création d'un fichier dans un premier conteneur . . . 18
5.1.2 Identification du conteneur . . . 19
5.1.3 Lecture du fichier avec `docker exec` . . . 19
5.1.4 Second conteneur Ubuntu . . . 19
5.1.5 Suppression du premier conteneur . . . 20
5.2 Partie V.2 — Création d'un volume . . . 20
5.2.1 Création . . . 20
5.2.2 Vérification . . . 20
5.3 Partie V.3 — Arrêt de l'ancien conteneur de l'application . . . 20
5.3.1 Affichage des conteneurs . . . 21
5.3.2 Suppression . . . 21
5.4 Partie V.4 — Démarrage de l'application avec le volume . . . 21
5.4.1 Lancement . . . 21
5.4.2 Ajout d'éléments à la liste . . . 21
5.4.3 Vérification des conteneurs . . . 22
5.5 Partie V.5 — Vérification de la persistance . . . 22
5.5.1 Identification du conteneur de l'application . . . 22
5.5.2 Suppression du conteneur . . . 22
5.5.3 Nouveau conteneur avec le même volume . . . 22
5.5.4 Éléments toujours présents . . . 23
5.6 Partie V.6 — Inspection du volume . . . 23

6 Devoir : construire, exécuter, partager et rendre persistante une image 24
6.1 Les étapes et les commandes . . . 24
6.2 Concepts à retenir . . . 25

7 Bilan des éléments créés 26

8 Difficultés rencontrées 27

Cadre du TP

Niveau	**Cours**	**Plateformes**
4e CY	Cloud Foundations	Docker Engine (Ubuntu 24.04) / Docker Hub / iximiuz Labs Docker Playground
Mode	**Approche**	**Livrable**
Groupe de 2 à 3	Tutoriel guidé *getting-started* exécuté dans un conteneur	Compte rendu + captures d'écran

TABLE 1 – Cadre général du TP

Objectif. Découvrir les **conteneurs Docker** en les expérimentant : exécuter un conteneur, **construire** une image à partir d'un `Dockerfile`, **partager** cette image sur un registre (Docker Hub) puis la relancer sur une machine qui ne l'a jamais vue, et enfin rendre **persistantes** les données créées par un conteneur grâce à un volume. Le support est une petite application de liste de tâches (*todo list*) en Node.js, fournie par le tutoriel *getting-started* de Docker.

Fil conducteur du TP :

Image getting-started → Dockerfile + `docker build` → `docker run` → Mise à jour de l'image → `docker tag` / `docker push` → Exécution sur le playground → Volume nommé

Environnement. Machine `Taha-inspiron-16` sous **Ubuntu 24.04** avec **Docker Engine** (pas de Docker Desktop, inutile sous Linux), compte **Docker Hub** `elmarocchino`, et playground en ligne **iximiuz Labs** pour la partie IV (le *Play with Docker* cité dans le tutoriel ne fonctionne plus). Le tutoriel *getting-started* est lui-même servi par un conteneur, consulté dans le navigateur.

1 Lancer l'image Docker getting-started

Partie I du TP : vérification de Docker, exécution de l'image du tutoriel et contrôle que le conteneur tourne.

1.1 Partie I.1 — Installation de Docker

Le sujet demande d'installer Docker Desktop. Sous Ubuntu, Docker est déjà installé sous forme de **Docker Engine** (le démon et son client en ligne de commande) : Docker Desktop, qui encapsule le moteur dans une petite machine virtuelle, n'est utile que sous Windows et macOS. La version installée est vérifiée avec `docker -version`.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
```

FIGURE 1 – Version de Docker installée sur la machine : Docker version 29.1.3 (build 29.1.3-0ubuntu3~24.04.2).

Un test préalable avec `docker run hello-world` a confirmé que le client communique bien avec le démon : le démon télécharge l'image `hello-world` depuis Docker Hub, crée un conteneur à partir d'elle et renvoie son message au terminal.

1.2 Partie I.2 — Exécution de l'image getting-started

1.2.1 Lancement du conteneur

La commande du sujet est `docker run -d -p 80:80 docker/getting-started`. Sur la machine, le port 80 de l'hôte était déjà occupé par un autre service : Docker a créé le conteneur mais n'a pas pu le démarrer (erreur *address already in use*). Le port côté hôte a donc été remplacé par **8080**, ce qui ne change rien au principe puisque le port du conteneur reste le port 80.

Listing 1 – Exécution de l'image du tutoriel (port 8080 de l'hôte)

```
1 docker run -d -p 8080:80 docker/getting-started
```

Élément	Signification
`docker run`	Crée un conteneur à partir d'une image et le démarre
`-d`	Mode détaché : le conteneur tourne en arrière-plan et le terminal reste libre
`-p 8080:80`	Redirection de port `hôte:conteneur` : le port 8080 de la machine est relié au port 80 du conteneur
`docker/getting-started`	Image utilisée, téléchargée depuis Docker Hub si elle n'est pas présente en local

TABLE 2 – Options de la commande de la Partie I

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4$ docker run -d -p 8080:80 docker/getting-started
387cdaae748edaf45c58149886d32872692f0efc5c42975c0e6951dc36741ff6
```

FIGURE 2 – Lancement de `docker run -d -p 8080:80 docker/getting-started` : Docker affiche l'identifiant complet du conteneur créé (`387cdaae748e...`).

1.2.2 Vérification

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED         STATUS         PORTS                                   NAMES
387cdaae748e   docker/getting-started   "/docker-entrypoint.…"   5 seconds ago   Up 4 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   distracted_kel
dysh
```

FIGURE 3 – `docker ps` : le conteneur `387cdaae748e` (image `docker/getting-started`, nom `distracted_keldysh`) est `Up` et expose `0.0.0.0:8080->80/tcp`.

Le conteneur est bien en cours d'exécution ; la démonstration à l'enseignant a été faite à ce stade.

2 Construire une image Docker

Partie II du TP : ouverture du tutoriel, puis réalisation de la section Our Application *: récupération du code, écriture du Dockerfile, construction de l'image et lancement d'un conteneur.*

2.1 Partie II.1 — Ouverture du tutoriel

Le tutoriel est servi par le conteneur de la Partie I ; il est ouvert dans le navigateur à l'adresse `http://localhost:8080/` (le port 8080 remplaçant le port 80 du sujet). La page affichée est une version traduite automatiquement en français par le navigateur ; ses sections (*Getting Started*, *Our Application*, *Updating our App*, *Sharing our App*, *Persisting our DB*, *Using Bind Mounts*, *Multi-Container Apps*, *Using Docker Compose*, *Image Building Best Practices*, *What Next ?*) figurent dans le menu de gauche.

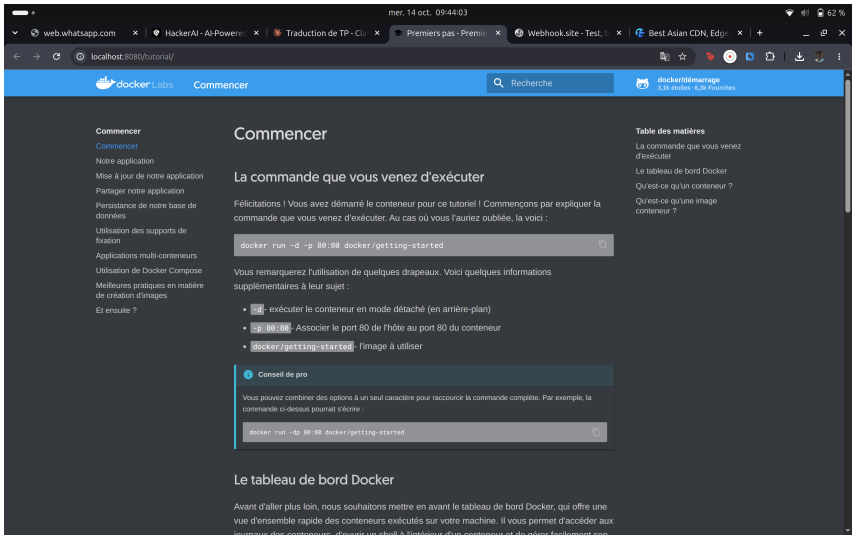


FIGURE 4 – Tutoriel *Docker Labs* servi par le conteneur sur `localhost:8080/tutorial/` : la page rappelle la commande exécutée et le rôle des options `-d`, `-p` et de l'image.

2.2 Partie II.2 et II.3 — Lecture de la section Getting Started

La section *Getting Started* a été lue rapidement (la vidéo n'est pas nécessaire pendant le TP). Elle explique la commande exécutée, le tableau de bord Docker, et définit les deux notions à retenir : un **conteneur** est un processus isolé de la machine hôte qui dispose de son propre système de fichiers, et une **image** est le modèle en lecture seule, composé de couches, à partir duquel on crée des conteneurs.

2.3 Partie II.4 — Section Our Application

2.3.1 Récupération du code source

Pour éviter un téléchargement dans le navigateur, l'archive `app.zip` de l'application a été récupérée directement depuis le conteneur du tutoriel, qui la sert dans son dossier `assets`.

Listing 2 – Récupération de l'archive de l'application depuis le conteneur du tutoriel

```
cd ~/Bureau/UIR/4A/S7/Cloud/TP4
curl -O http://localhost:8080/assets/app.zip
```

2.3.2 Décompression de l'application

Après décompression, le dossier `app` contient le fichier `package.json` (description du projet Node.js et de ses dépendances), le fichier `yarn.lock` (versions exactes des dépendances) et les deux sous-répertoires `src` (code de l'application) et `spec` (tests).

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4$ cd app
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ ls
package.json  spec  src  yarn.lock
```

FIGURE 5 – Contenu du dossier `app` après décompression : `package.json`, `spec`, `src` et `yarn.lock`.

2.3.3 Création du Dockerfile

Un **Dockerfile** est un script texte qui décrit, instruction par instruction, comment construire une image. Il est créé dans le dossier `app`, au même niveau que `package.json`.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ cat > Dockerfile <<'EOF'
FROM node:18-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
EOF
```

FIGURE 6 – Création du fichier `Dockerfile` dans le dossier `app` par une commande `cat > Dockerfile «'EOF'`, avec le contenu fourni par le tutoriel.



FIGURE 7 – Vérification avec `ls` : le fichier `Dockerfile` est présent à côté de `package.json`, sans extension `.txt`.

Instruction	**Rôle**
`FROM node:18-alpine`	Image de base : Node.js 18 sur Alpine Linux, une distribution très légère
`WORKDIR /app`	Dossier de travail dans l'image : les instructions suivantes s'y exécutent
`COPY . .`	Copie le contenu du dossier courant de l'hôte vers le dossier de travail de l'image
`RUN yarn install -production`	Installe, pendant la construction, les dépendances de production de l'application
`CMD ["node", "src/index.js"]`	Commande exécutée par défaut au démarrage d'un conteneur créé à partir de l'image

TABLE 3 – Instructions du Dockerfile de l'application

2.3.4 Construction de l'image

L'image est construite depuis le dossier `app`. La sortie de la commande est trop longue pour une capture d'écran : elle a été redirigée vers le fichier `docker_build_t_getting_started_partie_2.txt`, dont le contenu est reproduit ci-dessous.

Listing 3 – Construction de l'image

```
docker build -t getting-started .
```

Listing 4 – Sortie de la construction (fichier de la partie 2)

```
DEPRECATED: The legacy builder is deprecated and will be removed in
    a future release.
            Install the buildx component to build images with
               BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.626MB
Step 1/5 : FROM node:18-alpine
18-alpine: Pulling from library/node
25ff2da83641: Pulling fs layer
dd71dde834b5: Pulling fs layer
f18232174bc9: Pulling fs layer
1e5a4c89cee5: Pulling fs layer
25ff2da83641: Download complete
1e5a4c89cee5: Download complete
27e1cfd0b5f2: Download complete
0758d11b0bb7: Download complete
f18232174bc9: Download complete
```

```
f18232174bc9: Pull complete
dd71dde834b5: Download complete
25ff2da83641: Pull complete
1e5a4c89cee5: Pull complete
dd71dde834b5: Pull complete
Digest: sha256:8
   d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
Status: Downloaded newer image for node:18-alpine
 ---> 8d6421d663b4
Step 2/5 : WORKDIR /app
 ---> Running in ea5a23fcf5d2
 ---> Removed intermediate container ea5a23fcf5d2
 ---> a2d5bbe4e485
Step 3/5 : COPY . .
 ---> 6f1663c28c2a
Step 4/5 : RUN yarn install --production
 ---> Running in b4d88b980239
yarn install v1.22.22
[1/4] Resolving packages...
[2/4] Fetching packages...
[3/4] Linking dependencies...
[4/4] Building fresh packages...
Done in 6.64s.
 ---> Removed intermediate container b4d88b980239
 ---> 24a24cd36c2c
Step 5/5 : CMD ["node", "src/index.js"]
 ---> Running in 869cf67a145c
 ---> Removed intermediate container 869cf67a145c
 ---> 6dbeaf6a9c7d
Successfully built 6dbeaf6a9c7d
Successfully tagged getting-started:latest
```

La sortie se lit étape par étape, une étape par instruction du Dockerfile. L'image `node:18-alpine`, absente de la machine, est d'abord téléchargée couche par couche (`Pull complete`) ; chacune des instructions suivantes produit ensuite une nouvelle couche (identifiants de la forme `--> a2d5bbe4e485`). Le message `DEPRECATED` est un simple avertissement : l'ancien constructeur sera remplacé par BuildKit, mais la construction a réussi (`Successfully built 6dbeaf6a9c7d`). L'option `-t getting-started` donne à l'image un nom lisible, et le `.` final indique à Docker de chercher le `Dockerfile` dans le dossier courant, qui sert aussi de contexte de construction.

2.3.5 Lancement du conteneur de l'application

Listing 5 – Démarrage d'un conteneur à partir de l'image construite

```
docker run -dp 3000:3000 getting-started
```

L'option `-dp` regroupe `-d` (mode détaché) et `-p` (publication de port) ; `3000:3000` relie le port 3000 de l'hôte au port 3000 du conteneur, sans quoi l'application ne serait pas accessible depuis le navigateur.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker run -dp 3000:3000 getting-started
142a339fb50e7a1e9ed1509b3e957bd72b198a942f910c8aa245c18a6968875b
```

FIGURE 8 – Lancement de `docker run -dp 3000:3000 getting-started` : l'identifiant du conteneur (`142a339fb50e...`) est affiché.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS          PORTS                                              NAMES
142a339fb50e   getting-started        "docker-entrypoint.s…"   4 minutes ago    Up 4 minutes    0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp        elegant_
dijkstra
387cdaae748e   docker/getting-started   "/docker-entrypoint.…"   29 minutes ago   Up 29 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp            distract
ed_keldysh
```

FIGURE 9 – `docker ps` : deux conteneurs tournent, le conteneur de l'application (image `getting-started`, port 3000) et celui du tutoriel (image `docker/getting-started`, port 8080).

2.3.6 Test de l'application dans le navigateur

L'application est ouverte sur `http://localhost:3000`. Quelques éléments sont ajoutés à la liste, puis l'un d'eux est coché comme terminé et un autre est supprimé, pour vérifier que l'application fonctionne.

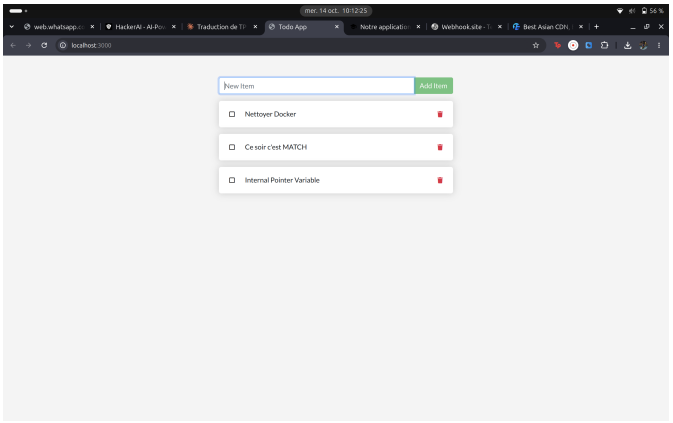


FIGURE 10 – Application sur `localhost:3000` après ajout de trois éléments à la liste.

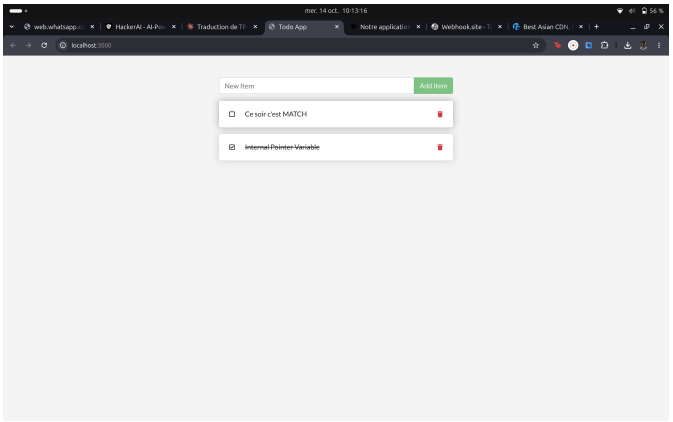


FIGURE 11 – Un élément a été supprimé (« Nettoyer Docker ») et un autre coché comme terminé (« Internal Pointer Variable », barré) : l'application fonctionne comme attendu.

2.3.7 Contrôle du Dockerfile final

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ cat Dockerfile
FROM node:18-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
```

FIGURE 12 – `cat Dockerfile` : contenu final du fichier, avec ses cinq instructions.

2.3.8 Image créée

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker images
                                                                    i Info →  U In Use
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/getting-started:latest  d79336f4812b       73.9MB         21.4MB    U
getting-started:latest         6dbeaf6a9c7d         342MB           84MB    U
hello-world:latest             11a3eb1597e8        25.9kB         9.49kB    U
node:18-alpine                 8d6421d663b4         181MB         45.3MB
```

FIGURE 13 – `docker images` : l'image `getting-started:latest` (ID `6dbeaf6a9c7d`) figure à côté de `docker/getting-started` (le tutoriel), de `hello-world` et de l'image de base `node:18-alpine`.

2.4 Partie II.5 — Vérification par l'enseignant

La section *Our Application* a été présentée à l'enseignant : l'application tourne sur `localhost:3000`.

3 Mettre à jour une image Docker

Partie III du TP : section Updating Our App. *L'équipe produit demande de modifier le texte affiché quand la liste est vide ; il faut reconstruire l'image et remplacer l'ancien conteneur.*

3.1 Partie III.1 — Modification du code source

Dans le fichier `src/static/js/app.js`, la ligne 56 est modifiée pour utiliser le nouveau texte (en anglais, comme dans le code d'origine).

Listing 6 – Modification de la ligne 56 de `src/static/js/app.js`

```
1 -                 <p className="text-center">No items yet! Add one
      above!</p>
2 +                 <p className="text-center">You have no todo items
      yet! Add one above!</p>
```

3.2 Partie III.2 — Reconstruction de l'image

L'image est reconstruite avec la même commande, depuis le dossier `app`. La sortie, redirigée vers le fichier `docker_build_t_getting_started_partie_3.txt`, est reproduite ci-dessous.

Listing 7 – Reconstruction de l'image

```
docker build -t getting-started .
```

Listing 8 – Sortie de la reconstruction (fichier de la partie 3)

```
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.629MB
Step 1/5 : FROM node:18-alpine
 ---> 8d6421d663b4
Step 2/5 : WORKDIR /app
 ---> Using cache
 ---> a2d5bbe4e485
Step 3/5 : COPY . .
 ---> 6c004425c02e
Step 4/5 : RUN yarn install --production
 ---> Running in 26973449bc54
yarn install v1.22.22
[1/4] Resolving packages...
[2/4] Fetching packages...
[3/4] Linking dependencies...
[4/4] Building fresh packages...
Done in 28.64s.
 ---> Removed intermediate container 26973449bc54
 ---> 198be8ae26c8
Step 5/5 : CMD ["node", "src/index.js"]
 ---> Running in 53c35f1b6390
 ---> Removed intermediate container 53c35f1b6390
 ---> b651a53f95bd
Successfully built b651a53f95bd
Successfully tagged getting-started:latest
```

Par rapport à la première construction, l'image de base n'est plus téléchargée et l'étape 2 indique `Using cache` : Docker réutilise les couches inchangées. À partir de l'étape 3, le contenu copié (`COPY . .`) n'est plus le même puisque `app.js` a été modifié, donc cette couche et toutes les suivantes sont reconstruites. La nouvelle image a pour identifiant `b651a53f95bd` et reprend le nom `getting-started:latest`.

3.3 Partie III.3 — Démarrage d'un nouveau conteneur : l'erreur attendue

Listing 9 – Tentative de démarrage du conteneur mis à jour

```
docker run -dp 3000:3000 getting-started
```

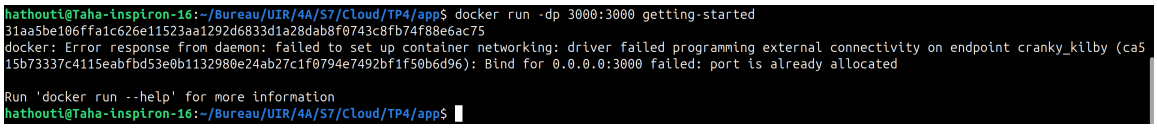


FIGURE 14 – Erreur `Bind for 0.0.0.0:3000 failed: port is already allocated` : le nouveau conteneur ne peut pas démarrer.

Explication. Le port 3000 de l'hôte est déjà utilisé par l'ancien conteneur, et un seul processus de la machine, conteneurs compris, peut écouter sur un port donné. Docker a créé un conteneur (`cranky_kilby`) mais n'a pas pu le démarrer. Une image est **immuable** : modifier le code ne change rien aux conteneurs existants, il faut supprimer l'ancien conteneur et en créer un nouveau à partir de la nouvelle image.

3.4 Partie III.4 — Remplacement de l'ancien conteneur

Pour supprimer un conteneur, il faut d'abord l'arrêter, puis le retirer.

3.4.1 Arrêt du conteneur

L'identifiant de l'ancien conteneur (`142a339fb50e`) est relevé avec `docker ps`, puis le conteneur est arrêté.

Listing 10 – Arrêt de l'ancien conteneur

```
1 docker ps
2 docker stop 142a339fb50e
```

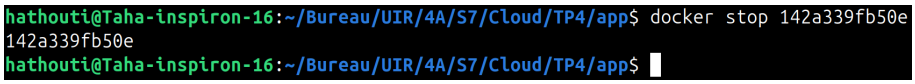


FIGURE 15 – `docker stop 142a339fb50e` : Docker renvoie l'identifiant du conteneur arrêté.



FIGURE 16 – `docker ps` : il ne reste que le conteneur du tutoriel sur le port 8080, le conteneur du port 3000 est arrêté.

3.4.2 Suppression du conteneur

Listing 11 – Suppression de l'ancien conteneur arrêté

```
1 docker rm 142a339fb50e
```

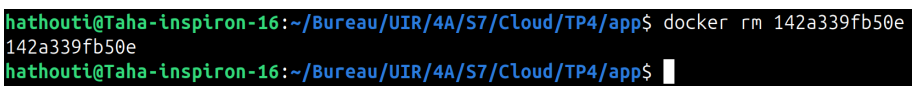


FIGURE 17 – `docker rm 142a339fb50e` : l'ancien conteneur est supprimé.

On peut aussi arrêter et supprimer un conteneur en une seule commande avec l'option de force : `docker rm -f <identifiant>`.

3.5 Partie III.5 — Démarrage du conteneur mis à jour

3.5.1 Lancement

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker run -dp 3000:3000 getting-started
2c5d28a3c4c687aa9d2156f17b5f40a780c7e20e2f4496b5cb231c35594eedb8
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$
```

FIGURE 18 – `docker run -dp 3000:3000 getting-started` : cette fois le conteneur démarre (identifiant `2c5d28a3c4c6...`).

3.5.2 Vérification

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS          PORTS                                       NAMES
2c5d28a3c4c6   getting-started        "docker-entrypoint.s…"   50 seconds ago   Up 50 seconds   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   strange_
sinoussi
387cdaae748e   docker/getting-started   "/docker-entrypoint.…"   5 hours ago      Up 5 hours      0.0.0.0:8080->80/tcp, [::]:8080->80/tcp       distract
ed_keldysh
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$
```

FIGURE 19 – `docker ps` : le nouveau conteneur `2c5d28a3c4c6` (image `getting-started`, port 3000) tourne, à côté du conteneur du tutoriel (port 8080).

3.6 Partie III.6 — Contrôle dans le navigateur

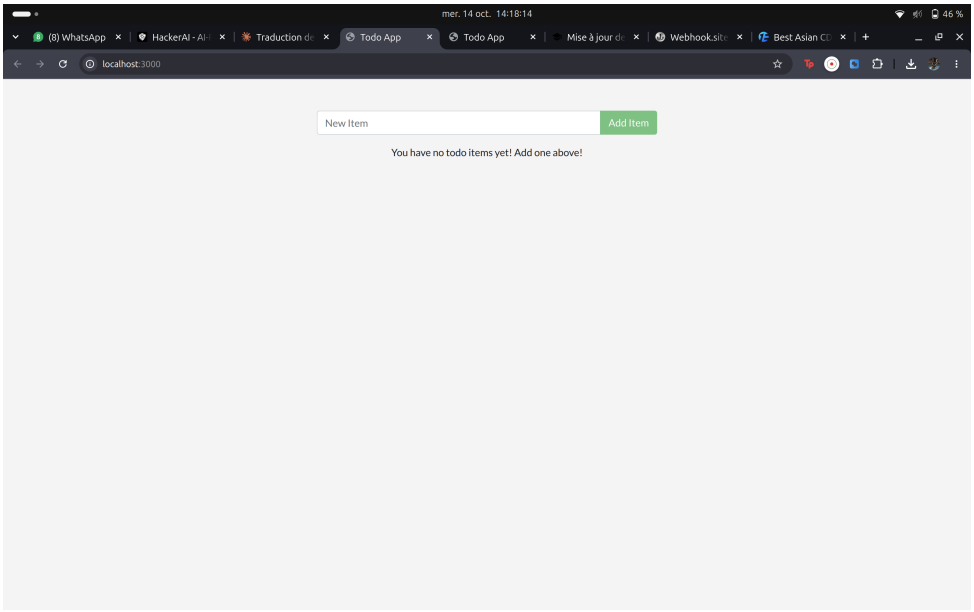


FIGURE 20 – Application rechargée sur `localhost:3000` : la liste est vide et affiche le nouveau message « You have no todo items yet ! Add one above ! ».

Le nouveau texte est bien affiché. En revanche, les éléments saisis précédemment ont disparu : leur base de données vivait dans l'ancien conteneur et a été détruite avec lui. C'est le problème traité à la Partie V. Cette partie a été montrée à l'enseignant.

4 Partager une image Docker

Partie IV du TP : section Sharing Our App. *L'image est publiée sur Docker Hub, puis exécutée sur une machine qui ne l'a jamais vue.*

4.1 Partie IV.1 — Dépôt sur Docker Hub

Un **registre** stocke et distribue des images ; le registre par défaut est **Docker Hub**, d'où viennent toutes les images utilisées jusqu'ici. Le dépôt `elmarocchino/getting-started` est créé sur le compte `elmarocchino`. L'onglet *Tags* indique *Pushed images appear here* : aucune image n'y a encore été envoyée.

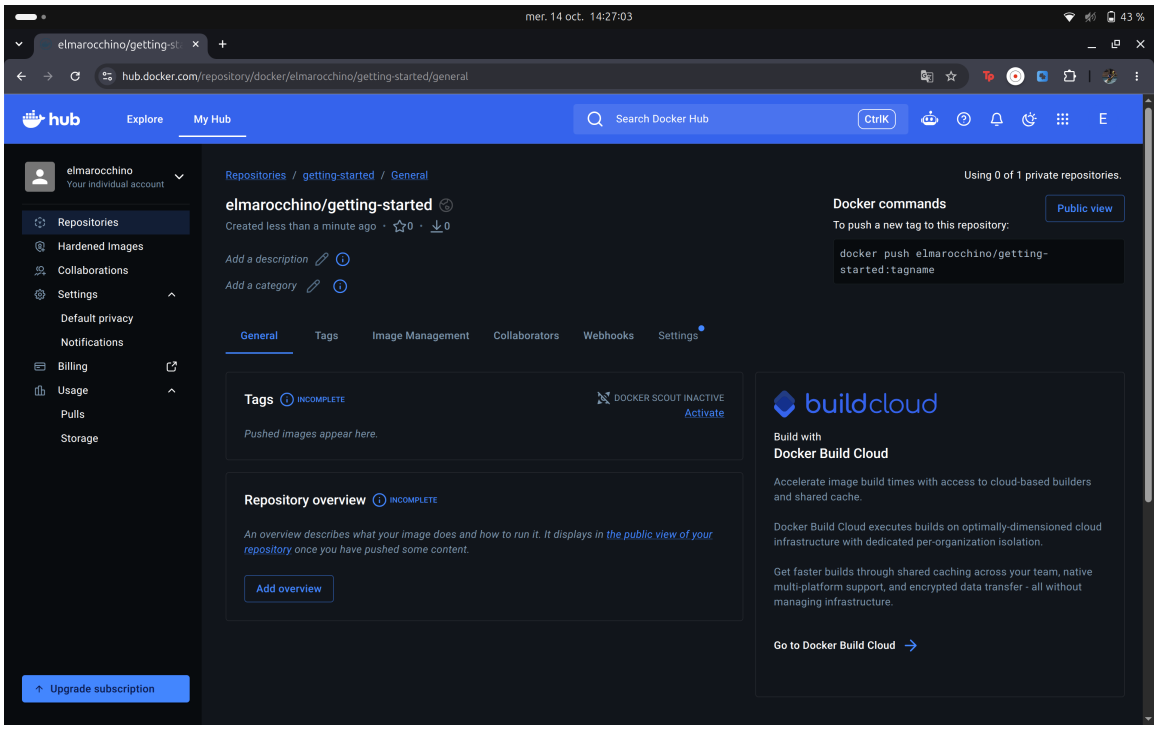


FIGURE 21 – Dépôt `elmarocchino/getting-started` sur Docker Hub, créé quelques instants plus tôt et encore vide. La section *Docker commands* indique la forme de la commande `docker push`.

4.2 Partie IV.2 — Connexion à Docker Hub

Listing 12 – Connexion au registre depuis le terminal

```
1 docker login -u elmarocchino
```

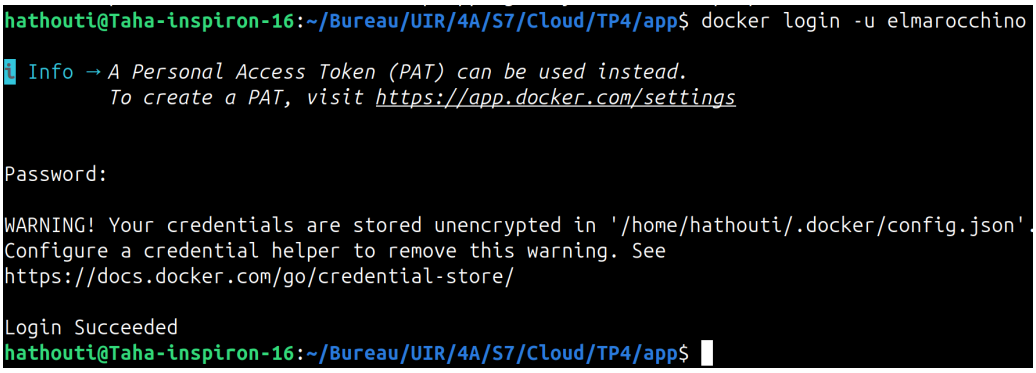


FIGURE 22 – `docker login -u elmarocchino` : `Login Succeeded`. Docker signale que les identifiants sont stockés non chiffrés dans `/home/hathouti/.docker/config.json`, et propose d'utiliser un jeton d'accès personnel (PAT).

4.3 Partie IV.3 — Premier essai de `docker push` : l'échec voulu

Listing 13 – Envoi de l'image sous son nom d'origine, avant tout étiquetage

```
1 docker push elmarocchino/getting-started
```

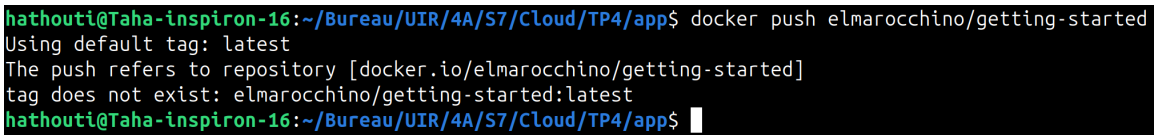


FIGURE 23 – `tag does not exist: elmarocchino/getting-started:latest` : la commande `push` cherche une image portant ce nom, et il n'en existe pas en local.

L'image locale s'appelle `getting-started` et non `elmarocchino/getting-started`. Sans étiquette (*tag*) précisée, Docker utilise `latest` (`Using default tag: latest`).

4.4 Partie IV.4 — Étiquetage de l'image

Listing 14 – Ajout d'un second nom à l'image

```
1 docker tag getting-started elmarocchino/getting-started
2 docker image ls
```

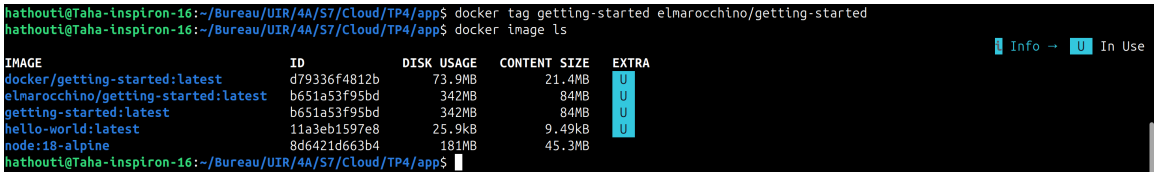


FIGURE 24 – `docker tag` puis `docker image ls` : `elmarocchino/getting-started:latest` et `getting-started:latest` portent le même identifiant `b651a53f95bd`, c'est-à-dire la même image (celle reconstruite en Partie III).

La commande `docker tag SOURCE CIBLE` ne copie pas l'image : elle lui ajoute un nom, de la forme `identifiant/dépôt[:étiquette]`.

4.5 Partie IV.5 — Envoi de l'image

Listing 15 – Envoi de l'image vers Docker Hub

```
docker push elmarocchino/getting-started
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker push elmarocchino/getting-started
Using default tag: latest
The push refers to repository [docker.io/elmarocchino/getting-started]
25ff2da83641: Mounted from library/node
1e5a4c89cee5: Mounted from library/node
dd71dde834b5: Mounted from library/node
f18232174bc9: Mounted from library/node
6c081150c29a: Pushed
6132729a57da: Pushed
dc0cdbb3637f: Pushed
latest: digest: sha256:b651a53f95bd2ab1d169facc33a5ed509c6cc1e77419172c95e50a9acbdf6bd2 size: 1838
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$
```

FIGURE 25 – `docker push elmarocchino/getting-started` : les quatre couches de `node` sont `Mounted from library/node` (déjà présentes sur Docker Hub), les trois couches propres à l'application sont `Pushed`, et le *digest* final est `sha256:b651a53f95bd...`.

Docker n'envoie que les couches que le registre ne possède pas déjà : celles de l'image de base `node:18-alpine` sont simplement rattachées, ce qui économise du transfert.

4.6 Partie IV.6 — Exécution sur une nouvelle instance : le playground

Le *Play with Docker* cité par le sujet ne fonctionne plus ; le playground Docker de **iximiuz Labs** (`labs.iximiuz.com/playgrounds/docker`) est utilisé à la place. Il fournit une machine Docker vierge, éphémère, sur laquelle l'image n'a jamais été téléchargée.

4.6.1 Démarrage du conteneur dans le playground

Listing 16 – Exécution de l'image publiée dans le playground

```
docker run -dp 3000:3000 elmarocchino/getting-started
```

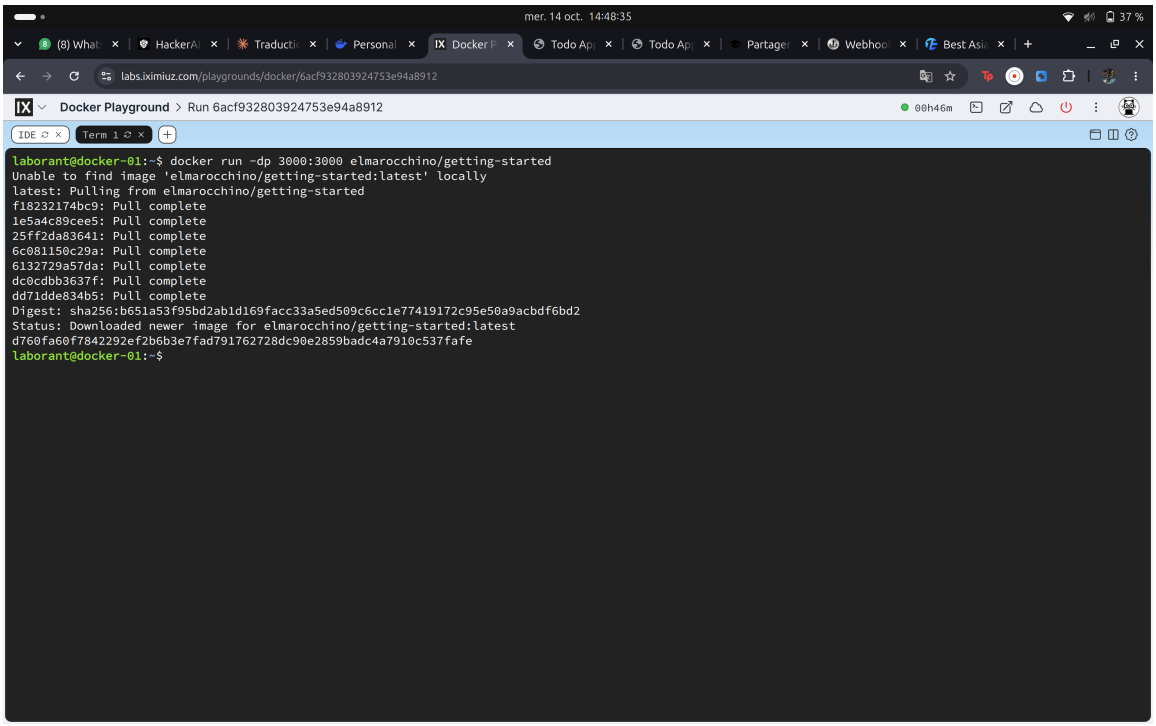


FIGURE 26 – Terminal du playground : l'image est introuvable en local (`Unable to find image`), donc téléchargée depuis Docker Hub (`Pull complete` pour chaque couche). Le *digest* `sha256:b651a53f95bd...` est identique à celui du `push`, puis le conteneur démarre.

Le téléchargement sans authentification confirme aussi que le dépôt est bien public.

4.6.2 Exposition du port 3000

Le playground ne propose pas de badge de port : le port 3000 est exposé depuis la fenêtre *Expose HTTP(S) Ports*, en saisissant `3000` puis en cliquant sur *EXPOSE*.

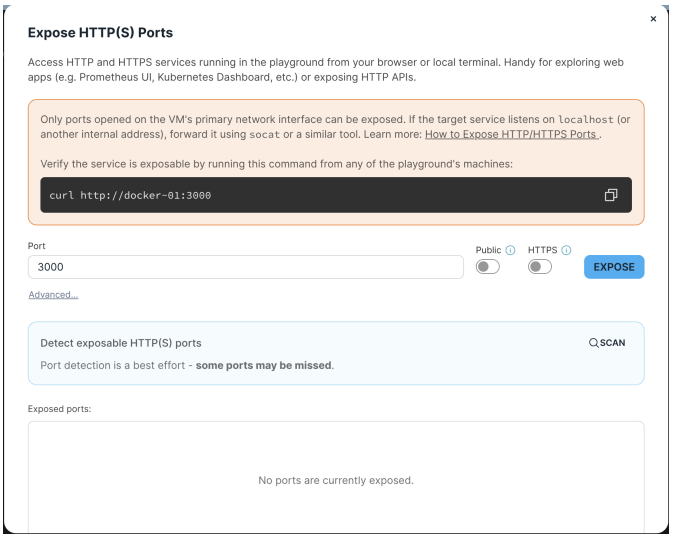


FIGURE 27 – Fenêtre *Expose HTTP(S) Ports* du playground : le port `3000` est saisi avant d'être exposé. Le service est testable depuis la machine avec `curl http://docker-01:3000`.

4.6.3 Application ouverte depuis le playground

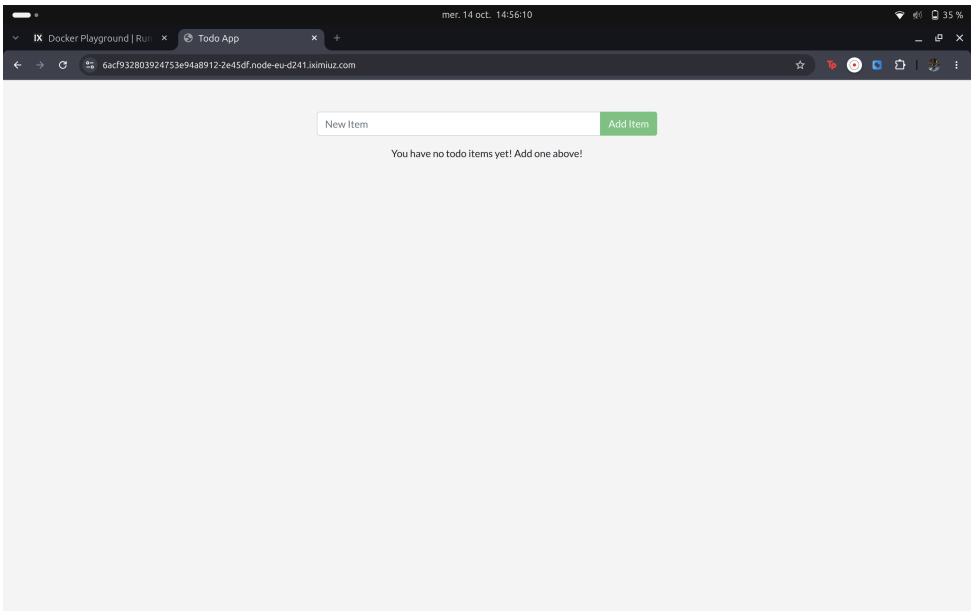


FIGURE 28 – Application ouverte depuis l'adresse publique fournie par le playground (`...node-eu-d241.iximiuz.com`) : elle affiche le nouveau message « You have no todo items yet ! Add one above ! », ce qui prouve que c'est bien la version modifiée qui a été publiée.

La liste est vide : c'est un conteneur tout neuf, avec sa propre base de données. Cette pratique est courante en intégration continue : un *pipeline* construit l'image, l'envoie au registre, et l'environnement de production télécharge la dernière version. Cette partie a été montrée à l'enseignant.

5 Rendre persistantes les données créées par un conteneur

Partie V du TP : section Persisting Our DB. *On constate d'abord que chaque conteneur dispose de son propre système de fichiers, puis on utilise un volume nommé pour conserver la base de données.*

5.1 Partie V.1 — Le système de fichiers d'un conteneur

Un conteneur utilise les couches de son image pour son système de fichiers, plus un espace de travail temporaire propre pour créer, modifier et supprimer des fichiers. Ces modifications ne sont pas visibles dans un autre conteneur, même créé à partir de la même image. L'expérience qui suit le démontre avec deux conteneurs `ubuntu`.

5.1.1 Création d'un fichier dans un premier conteneur

Listing 17 – Conteneur Ubuntu qui écrit un nombre aléatoire dans /data.txt

```
docker run -d ubuntu bash -c "shuf -i 1-10000 -n 1 -o /data.txt && tail -f /dev/null"
```

La commande lance un shell `bash` qui exécute deux commandes : `shuf` écrit un nombre aléatoire entre 1 et 10000 dans le fichier `/data.txt`, puis `tail -f /dev/null` ne fait rien mais empêche le conteneur de s'arrêter.

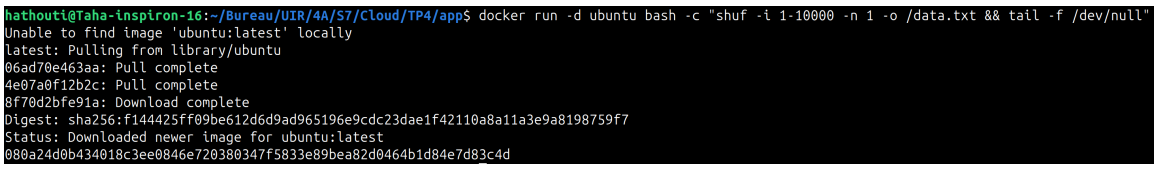


FIGURE 29 – Lancement du conteneur Ubuntu : l'image `ubuntu:latest` est téléchargée, puis l'identifiant du conteneur (`080a24d0b434...`) est affiché.

5.1.2 Identification du conteneur

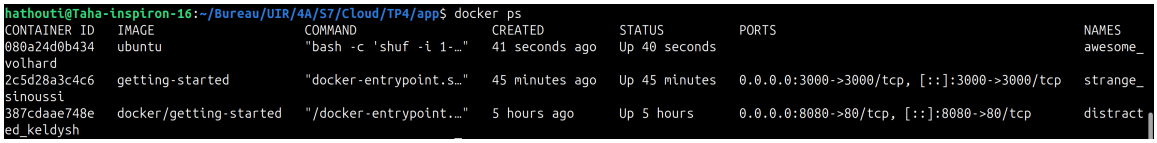


FIGURE 30 – `docker ps` : le conteneur Ubuntu `080a24d0b434` (nom `awesome_volhard`) tourne, avec les conteneurs de l'application et du tutoriel.

5.1.3 Lecture du fichier avec `docker exec`

Listing 18 – Lecture du fichier dans le conteneur en cours d'exécution

```
docker exec 080a24d0b434 cat /data.txt
```



FIGURE 31 – `docker exec 080a24d0b434 cat /data.txt` : le fichier existe et contient le nombre `9905`.

La commande `docker exec` exécute une commande dans un conteneur déjà en cours d'exécution.

5.1.4 Second conteneur Ubuntu

Listing 19 – Second conteneur à partir de la même image

```
docker run -it ubuntu ls /
```

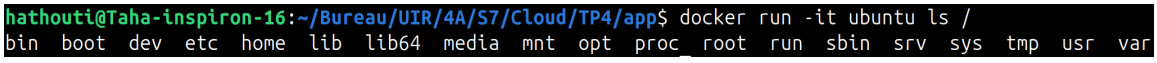


FIGURE 32 – `docker run -it ubuntu ls /` : le contenu de la racine ne contient pas de fichier `data.txt`. L'option `-it` alloue un terminal interactif.

Le fichier `data.txt` n'a été écrit que dans l'espace de travail du premier conteneur : le second, créé à partir de la même image, ne le voit pas.

5.1.5 Suppression du premier conteneur

Listing 20 – Suppression du conteneur Ubuntu

```
1 docker rm -f 080a24d0b434
```



FIGURE 33 – `docker rm -f 080a24d0b434` : le premier conteneur est supprimé ; le fichier `data.txt` disparaît avec lui.

5.2 Partie V.2 — Création d'un volume

Les modifications d'un conteneur sont perdues à sa suppression, car elles lui sont isolées. Un **volume** relie un chemin du système de fichiers du conteneur à un espace de stockage géré par Docker sur l'hôte, indépendant du cycle de vie des conteneurs. Un **volume nommé** est un simple espace de stockage identifié par son nom : Docker gère l'emplacement physique sur le disque.

5.2.1 Création

Listing 21 – Création du volume todo-db

```
1 docker volume create todo-db
```



FIGURE 34 – `docker volume create todo-db` : Docker renvoie le nom du volume créé.

5.2.2 Vérification



FIGURE 35 – `docker volume ls` : le volume `todo-db` existe, avec le pilote `local`.

5.3 Partie V.3 — Arrêt de l'ancien conteneur de l'application

Le conteneur de l'application encore en cours d'exécution doit être supprimé, car il occupe le port 3000 et n'utilise pas le volume persistant.

5.3.1 Affichage des conteneurs

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED              STATUS              PORTS
NAMES
82de4ac7c639   docker/getting-started   "/docker-entrypoint.…"   About a minute ago   Up About a minute   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
clever_lehmann
54b648691e1a   getting-started          "docker-entrypoint.s…"   2 minutes ago        Up 2 minutes        0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
nostalgic_northcutt
```

FIGURE 36 – `docker ps` : le conteneur de l'application (`54b648691e1a`, image `getting-started`, port 3000) est à supprimer ; celui du tutoriel (`82de4ac7c639`, port 8080) est conservé.

5.3.2 Suppression

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker rm -f 54b648691e1a
54b648691e1a
```

FIGURE 37 – `docker rm -f 54b648691e1a` : le conteneur de l'application est arrêté puis supprimé en une seule commande.

5.4 Partie V.4 — Démarrage de l'application avec le volume

5.4.1 Lancement

Listing 22 – Démarrage de l'application avec le volume monté sur /etc/todos

```
1 docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started
```

L'option `-v todo-db:/etc/todos` se lit `nom_du_volume:chemin_dans_le_conteneur`. L'application stocke ses données dans une base SQLite, un fichier unique `/etc/todos/todo.db` : en montant le volume sur ce dossier, le fichier est écrit dans le volume, sur l'hôte, et non plus dans l'espace temporaire du conteneur.

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started
dcd7cf5c2fd9cef17fa9120cbd82ee504ed907359053f5704f45fa878881d3b0
```

FIGURE 38 – Lancement du conteneur de l'application avec le volume `todo-db` monté sur `/etc/todos` (identifiant `dcd7cf5c2fd9...`).

5.4.2 Ajout d'éléments à la liste

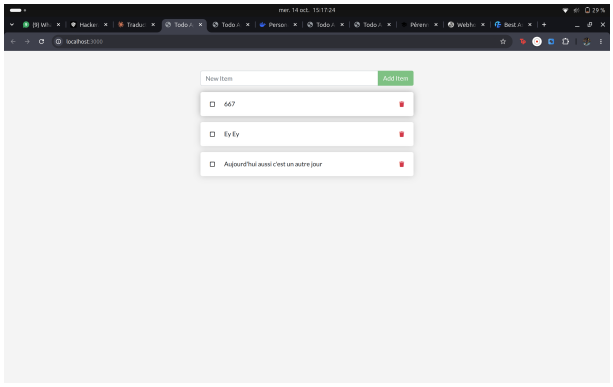


FIGURE 39 – Application sur `localhost:3000` : trois éléments (« 667 », « Ey Ey » et « Aujourd'hui aussi c'est un autre jour ») ont été ajoutés à la liste.

5.4.3 Vérification des conteneurs

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS          PORTS                                         NAMES
dcd7cf5c2fd9   getting-started        "docker-entrypoint.s…"   13 seconds ago   Up 12 seconds   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   pedantic
_hypatia
82de4ac7c639   docker/getting-started   "/docker-entrypoint.…"   3 minutes ago    Up 3 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp       clever_l
ehmann
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$
```

FIGURE 40 – `docker ps` : le conteneur `dcd7cf5c2fd9` (application, port 3000) et celui du tutoriel (port 8080) tournent.

5.5 Partie V.5 — Vérification de la persistance

On détruit le conteneur de l'application, puis on en démarre un nouveau avec la même commande : les éléments doivent être toujours présents.

5.5.1 Identification du conteneur de l'application

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS          PORTS                                         NAMES
dcd7cf5c2fd9   getting-started        "docker-entrypoint.s…"   3 minutes ago    Up 3 minutes    0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   pedantic_h
ypatia
82de4ac7c639   docker/getting-started   "/docker-entrypoint.…"   6 minutes ago    Up 6 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp       clever_leh
mann
```

FIGURE 41 – `docker ps` : l'identifiant du conteneur de l'application est `dcd7cf5c2fd9` (image `getting-started`, port 3000) ; le conteneur `82de4ac7c639`, image `docker/getting-started`, est celui du tutoriel.

5.5.2 Suppression du conteneur

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker rm -f dcd7cf5c2fd9
dcd7cf5c2fd9
```

FIGURE 42 – `docker rm -f dcd7cf5c2fd9` : le conteneur de l'application est supprimé, et avec lui son système de fichiers temporaire.

5.5.3 Nouveau conteneur avec le même volume

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started
ad81970b9b9bc14f50efa98f1ef94f18c2544bc4381f691d61e6fc225c32947e
```

FIGURE 43 – Même commande `docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started` : un nouveau conteneur (`ad81970b9b9b...`) est créé et monte le volume existant.

5.5.4 Éléments toujours présents

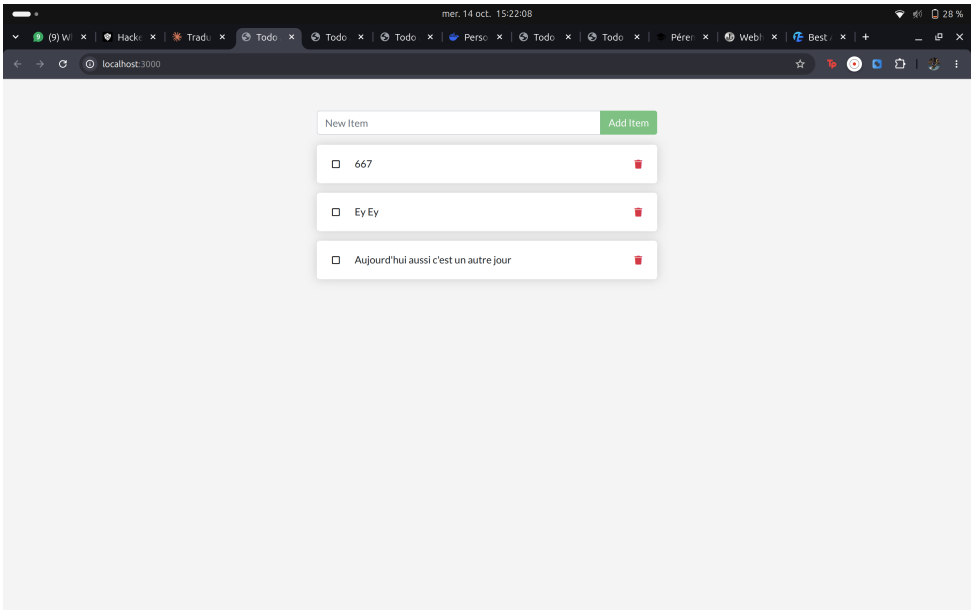


FIGURE 44 – Page rechargée sur `localhost:3000` : les trois éléments ajoutés avant la destruction du conteneur sont toujours dans la liste.

C'est la preuve de la persistance : les données ont survécu à la suppression du conteneur, car la base `todo.db` est stockée dans le volume. À l'inverse, lors de la Partie III, la liste était vide après le remplacement du conteneur, faute de volume.

5.6 Partie V.6 — Inspection du volume

Listing 23 – Localisation du volume sur le disque

```
1 docker volume inspect todo-db
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cloud/TP4/app$ docker volume inspect todo-db
[
    {
        "CreatedAt": "2026-10-14T15:04:25Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/snap/docker/common/var-lib-docker/volumes/todo-db/_data",
        "Name": "todo-db",
        "Options": null,
        "Scope": "local"
    }
]
```

FIGURE 45 – `docker volume inspect todo-db` : pilote `local`, portée `local`, et `Mountpoint` indiquant l'emplacement des données sur le disque de l'hôte (`/var/snap/docker/common/var-lib-docker/volumes/todo-db/_data`).

Le champ `Mountpoint` est l'emplacement précis où Docker stocke les données du volume ; il se trouve dans l'arborescence de Docker et son accès demande en général les droits d'administrateur. Cette partie a été montrée à l'enseignant.

6 Devoir : construire, exécuter, partager et rendre persistante une image

6.1 Les étapes et les commandes

1. **Écrire le Dockerfile** dans le dossier du code : image de base, dossier de travail, copie du code, installation des dépendances et commande de démarrage.
2. **Construire l'image** avec `docker build -t getting-started .`, depuis le dossier qui contient le Dockerfile.
3. **Exécuter un conteneur** avec `docker run -dp 3000:3000 getting-started`, puis le vérifier avec `docker ps` et le navigateur.
4. **Mettre à jour** : modifier le code, reconstruire l'image, supprimer l'ancien conteneur (`docker stop` puis `docker rm`, ou `docker rm -f`) et en lancer un nouveau.
5. **Partager** : se connecter (`docker login`), étiqueter l'image (`docker tag`), l'envoyer (`docker push`), puis la lancer sur une autre machine (`docker run` depuis le playground).
6. **Rendre persistante** : créer un volume (`docker volume create`) et le monter sur le dossier de données avec `-v`.

Commande ou option	Rôle
`docker build -t getting-started .`	Construit une image à partir du Dockerfile du dossier courant (`.`) et la nomme `getting-started` (`-t`)
`docker run`	Crée et démarre un conteneur à partir d'une image
`-d`	Mode détaché : exécution en arrière-plan
`-p 3000:3000`	Relie le port 3000 de l'hôte au port 3000 du conteneur (`hôte:conteneur`)
`-v todo-db:/etc/todos`	Monte le volume nommé `todo-db` sur le dossier `/etc/todos` du conteneur
`-it`	Terminal interactif (`-i` garde l'entrée ouverte, `-t` alloue un pseudo-terminal)
`docker ps`	Liste les conteneurs en cours d'exécution
`docker images` / `docker image ls`	Liste les images locales
`docker stop <id>`	Arrête proprement un conteneur
`docker rm <id>` / `docker rm -f <id>`	Supprime un conteneur arrêté / force l'arrêt puis la suppression
`docker exec <id> <commande>`	Exécute une commande dans un conteneur en cours d'exécution
`docker login -u <identifiant>`	Authentifie le client auprès du registre (Docker Hub)
`docker tag SOURCE CIBLE`	Ajoute un second nom (`identifiant/dépôt[:étiquette]`) à une image, sans la copier
`docker push <nom>`	Envoie les couches de l'image vers le registre
`docker volume create` / `ls` / `inspect`	Crée, liste et décrit (dont l'emplacement, `Mountpoint`) un volume

TABLE 4 – Commandes et options utilisées dans le TP

6.2 Concepts à retenir

— Une **image** est un modèle en lecture seule, formé de couches, identifié par un nom et un identifiant. Un **conteneur** est une instance en cours d'exécution d'une image, avec son propre système de fichiers temporaire.

— Une image est **immuable** : pour mettre à jour une application, on reconstruit l'image, on supprime l'ancien conteneur et on en lance un nouveau. Le **cache** de construction réutilise les couches inchangées et accélère les reconstructions.

— Un **registre** comme Docker Hub permet de partager des images : le nom `identifiant/dépôt` associe l'image à un compte, et `latest` est l'étiquette par défaut.

— Les données écrites dans un conteneur sont **perdues** à sa suppression. Un **volume nommé** les conserve sur l'hôte, indépendamment des conteneurs, et peut être remonté dans le suivant.

7 Bilan des éléments créés

Élément	**Rôle**
Image `docker/getting-started`	Tutoriel Docker, servi sur le port 8080
Image `getting-started` (`6dbeaf6a9c7d`, puis `b651a53f95bd` après mise à jour)	Image de l'application de liste de tâches, construite à partir du Dockerfile
Image `elmarocchino/getting-started`	Second nom de l'image mise à jour, publiée sur Docker Hub
Images `node:18-alpine`, `ubuntu`, `hello-world`	Image de base de l'application, expérience sur le système de fichiers, test d'installation
Conteneur d'application (port 3000)	Exécute l'application ; remplacé à chaque mise à jour, puis lancé avec le volume
Volume `todo-db`	Stocke la base SQLite `todo.db` de l'application, indépendamment des conteneurs
Dépôt Docker Hub `elmarocchino/getting-started`	Registre public où l'image est partagée

TABLE 5 – Éléments Docker impliqués dans le TP4

8 Difficultés rencontrées

Problème	Cause	Solution
`docker run -d -p 80:80 docker/getting-started` échoue avec *address already in use*	Le port 80 de la machine est déjà utilisé par un autre service	Utiliser un autre port côté hôte : `-p 8080:80`, puis ouvrir `localhost:8080`
`docker build` lancé depuis `app/src/static/js` : *unable to prepare context* et *Dockerfile : no such file or directory*	Le `.` désigne le dossier courant, qui ne contient pas le Dockerfile	Se placer dans le dossier `app` avant de construire
port is already allocated au lancement du conteneur mis à jour	L'ancien conteneur occupe encore le port 3000 de l'hôte	Arrêter et supprimer l'ancien conteneur, puis relancer
`docker push` : *tag does not exist*	L'image locale n'a pas le nom `identifiant/dépôt`	Étiqueter l'image avec `docker tag getting-started elmarocchino/getting-started`
`docker push` : *authentication required - access token has insufficient scopes*	Connexion faite avec un jeton d'accès personnel sans droit d'écriture (cause probable)	Se reconnecter avec un jeton ayant les droits de lecture et d'écriture (ou le mot de passe du compte)
Playground : *repository name must be lowercase*, puis *not found*	Le texte `TON_ID` avait été copié tel quel ; ensuite l'image n'était pas encore publiée sur Docker Hub	Utiliser le vrai identifiant en minuscules, et relancer `docker run` une fois le `push` réussi
Playground : *port is already allocated* lors d'un second `docker run`	Le premier `docker run` avait déjà démarré le conteneur sur le port 3000	Ne pas relancer : vérifier avec `docker ps` et supprimer le conteneur raté
Suppression par erreur du conteneur du tutoriel (port 8080) en Partie V	Mauvais identifiant utilisé avec `docker rm -f`	Relancer `docker run -d -p 8080:80 docker/getting-started` (nouveau conteneur `82de4ac7c639`) et supprimer le conteneur de l'application

Table 6 – Problèmes rencontrés et solutions